

《优化方法基础》课程作业

等式约束优化问题

姓名：林圣翔 班级：计试 2201 学号：2223312202

2025 年 11 月 1 日

问题 1.

$$\begin{aligned} \min_{x \in \mathbb{R}^n} f(x) \\ \text{s.t. } Ax = b \end{aligned}$$

其中：

- $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 为二次连续可微凸函数；
- $A \in \mathbb{R}^{p \times n}$, $\text{rank}(A) = p < n$ 。

总结等式约束凸优化问题的求解方法及相互联系

解答.

1. KKT 方程方法

拉格朗日函数为 $L(x, v) = f(x) + v^T(Ax - b)$, KKT 条件为：

$$\begin{aligned} Ax^* &= b \\ \nabla f(x^*) + A^T v^* &= 0 \end{aligned}$$

直接求解 KKT 方程可得最优解 (x^*, v^*) 。该方法将优化问题转化为非线性方程组求解，但当方程复杂时，求解可能困难。

2. 对偶方法

对偶函数为 $g(v) = \min_x f(x) + v^T(Ax - b)$, 对偶问题为 $\max_v g(v)$ 。求解对偶问题与求解 KKT 方程本质相同，但对偶方法采用优化算法（如梯度上升）求解，避免直接解方程，更适用于大规模问题。

3. 消除方法

通过变量消除将约束问题转化为无约束问题。令 $x = Fz + \hat{x}$, 其中 $F \in \mathbb{R}^{n \times (n-p)}$ 满足 $AF = 0$, $\hat{x} \in \mathbb{R}^n$ 满足 $A\hat{x} = b$ 。具体构造为：

$$F = P \begin{bmatrix} -A_1^{-1}A_2 \\ I \end{bmatrix}, \quad \hat{x} = P \begin{bmatrix} A_1^{-1}b \\ 0 \end{bmatrix}$$

其中 P 是初等列变换矩阵，使得 A_1 (A 的前 p 列) 可逆。原问题变为 $\min_z f(Fz + \hat{x})$, 可用无约束优化方法（如牛顿法）求解。

4. 可行初始点的牛顿迭代

从可行点 x^0 (即 $Ax^0 = b$) 开始, 每次迭代求解牛顿方向 d_x 来自:

$$\begin{bmatrix} \nabla^2 f(x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} d_x \\ w \end{bmatrix} = \begin{bmatrix} -\nabla f(x) \\ 0 \end{bmatrix}$$

然后通过线搜索更新 $x^{k+1} = x^k + td_x^k$, 确保迭代点始终可行。收敛条件基于牛顿减少量 $\lambda(x)^2 = d_x^T \nabla^2 f(x) d_x$ 。

5. 一般初始点的牛顿迭代

从不可行点开始, 同时更新原变量和对偶变量。定义残差 $r(y) = r(x, v) = \begin{bmatrix} \nabla f(x) + A^T v \\ Ax - b \end{bmatrix}$, 牛顿方向 (d_x, d_v) 来自:

$$\begin{bmatrix} \nabla^2 f(x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} d_x \\ d_v \end{bmatrix} = \begin{bmatrix} -A^T v - \nabla f(x) \\ b - Ax \end{bmatrix}$$

更新 $x^{k+1} = x^k + td_x^k$, $v^{k+1} = v^k + td_v^k$, 以减少 $\|r(y)\|_2$ 。该方法最终使迭代点收敛到可行点。

方法间的相互联系

1. 消除方法与可行牛顿方法

在消除方法中, 牛顿方向为: $d_z = -(F^T \nabla^2 f(x) F)^{-1} F^T \nabla f(x)$

对应原空间方向: $d_x = F d_z = -F (F^T \nabla^2 f(x) F)^{-1} F^T \nabla f(x)$

构造: $w = (AA^T)^{-1} A (\nabla f(x) + \nabla^2 f(x) d_x)$

则有 $\begin{bmatrix} F^T \\ A \end{bmatrix} (\nabla^2 f(x) d_x + A^T w + \nabla f(x)) = 0$ 满足 KKT 系统

$\therefore \tilde{\lambda}(z)^2 = d_z^T \nabla^2 \hat{f}(z) d_z = d_z^T F^T \nabla^2 f(x) F d_z = d_x^T \nabla^2 f(x) d_x = \lambda(x)^2$

$\therefore$ 牛顿减少量相同

综上, 消除方法与可行牛顿方法等价。

2. 可行牛顿方法与一般初始点的牛顿法

一般初始点的牛顿法在迭代过程中, 残差 $r_{\text{pri}}(y) = Ax - b$ 会逐渐减小, 最终收敛到可行点 (如公式 $r_{\text{pri}}^{k-1} = \prod_{i=0}^{k-1} (1 - t^i) r_{\text{pri}}^0$ 所示)。此时, 系统退化为可行牛顿法的形式, 即 d_v 的更新相当于对偶变量 w 的调整。因此, 一般初始点的牛顿法是可行牛顿法的扩展, 处理不可行起点。

3. 对偶方法与 KKT 方程

对偶方法求解 $\max_v g(v)$, 其最优条件与 KKT 方程一致。对偶方法通过优化对偶变量间接求解原问题, 而 KKT 方程直接求解系统。对偶方法更适合当对偶问题更容易求解时。

问题 2. 等式约束熵极大化。考虑等式约束熵极大化问题:

$$\begin{aligned} \text{minimize} \quad & f(x) = \sum_{i=1}^n x_i \log x_i \\ \text{subject to} \quad & Ax = b \end{aligned}$$

其中 $\text{dom } f = \mathbb{R}_{++}^n$, $A \in \mathbb{R}^{p \times n}$, $p < n$ 。(一些相关分析见习题 10.9。) 生成一个 $n = 100$, $p = 30$ 的问题实例, 随机选择 A (验证其为满秩阵), 随机选择一个正向量作为 $\hat{x}$ (例如, 其分量在区间 $[0, 1]$ 上均匀分布), 然后令 $b = A\hat{x}$ 。(于是, $\hat{x}$ 可行。) 采用以下方法计算该问题的解。

- (a) **标准 Newton 方法**。可以选用初始点 $x^{(0)} = \hat{x}$ 。
- (b) **不可行初始点 Newton 方法**。可以选用初始点 $x^{(0)} = \hat{x}$ (和标准 Newton 方法比较), 也可以选用初始点 $x^{(0)} = 1$ 。
- (c) **对偶 Newton 方法**, 即将标准 Newton 方法应用于对偶问题。证实三种方法求得相同的最优点 (和 Lagrange 乘子)。
- 比较三种方法每步迭代的计算量, 假设利用了相应的结构。(但在你的实现中不需要利用结构计算 Newton 步径。)

解答.

1. 问题实例生成

根据题意, 生成了问题实例, 满足如下条件:

- 1) $n = 100$, $p = 30$;
- 2) 矩阵 $A \in \mathbb{R}^{30 \times 100}$ 随机生成, 通过验证为满秩矩阵 (秩 = 30);
- 3) $\hat{x}$ 为随机生成的正向量, 各分量在区间 $[0, 1]$ 上均匀分布, 经检验, 随机生成的数据分布于 $[0.0327, 0.9836]$;
- 4) 经检验, $b = A\hat{x}$, 确保 $\hat{x}$ 是可行点

2. 求解方法 (具体函数及其功能详见表 1)

2.1. 标准 Newton 方法

标准牛顿法用于求解带约束的优化问题, 核心是构建并求解 KKT 系统, 结合线搜索确定合适步长迭代更新解。

目标函数为 $f(x) = \sum_{i=1}^n x_i \log(x_i)$, 其梯度 $\nabla f(x)$ 满足 $(\nabla f(x))_i = \log(x_i) + 1$, Hessian 矩阵 $\nabla^2 f(x)$ 为对角矩阵, 即 $(\nabla^2 f(x))_{ii} = \frac{1}{x_i}$ (对应代码中 `objective`、`gradient`、`hessian` 函数)。

$$\text{构建 KKT 矩阵 } \text{KKT_matrix} = \begin{bmatrix} \nabla^2 f(x) & A^T \\ A & \mathbf{0}_{p \times p} \end{bmatrix} \quad \text{右侧项 rhs} = \begin{bmatrix} -\nabla f(x) \\ b - Ax \end{bmatrix}$$

通过求解 $\text{KKT_matrix} \cdot \text{step} = \text{rhs}$ 得到步长相关量 step , 其中前 n 个元素为 dx (解的更新步长), 后 p 个元素为拉格朗日乘子 λ_{vals} (对应代码中构建 KKT 矩阵和求解的部分)。

采用回溯线搜索确定步长 t , 满足:

$$\begin{cases} x + t \cdot dx > 0 & (\text{保证变量非负, 因目标函数定义域要求}) \\ f(x + t \cdot dx) \leq f(x) + \alpha \cdot t \cdot \nabla f(x)^T \cdot dx \end{cases}$$

其中 $\alpha = 0.3$, $\beta = 0.5$ 为线搜索参数, 迭代更新 $t = t \cdot \beta$ 直到满足条件或 t 过小 (对应代码中线搜索循环部分)。

当梯度范数 $\|\nabla f(x) + A^T \lambda_{\text{vals}}\| < \text{tol}$ 且约束违反程度 $\|Ax - b\|_2 < \text{tol}$ 时, 停止迭代, 其中 $\text{tol} = 10^{-8}$ (对应代码中迭代终止判断部分)。

2.2. 不可行初始点 Newton 方法

允许从不可行初始点开始迭代, 通过构建残量相关的 KKT 系统, 同时更新解 x 和拉格朗日乘子 v 来逐步满足约束和优化条件。

定义对偶残量 $r_{\text{dual}} = \nabla f(x) + A^T v$, 原问题残量 $r_{\text{pri}} = Ax - b$ (对应代码中计算 r_{dual} 和 r_{pri} 的部分)。

构建与残量相关的 KKT 矩阵 (同标准牛顿法的 KKT 矩阵形式) $\text{KKT_matrix} = \begin{bmatrix} \nabla^2 f(x) & A^T \\ A & \mathbf{0}_{p \times p} \end{bmatrix}$

右侧项 $\text{rhs} = - \begin{bmatrix} r_{\text{dual}} \\ r_{\text{pri}} \end{bmatrix}$

求解 $\text{KKT_matrix} \cdot \text{step} = \text{rhs}$ 得到 dx (解的更新步长) 和 dv (拉格朗日乘子的更新步长) (对应代码中构建 KKT 矩阵和求解的部分)。

基于残量范数进行回溯线搜索, 残量范数定义为 $\text{residual_norm}(x, v) = \left\| \begin{bmatrix} r_{\text{dual}} \\ r_{\text{pri}} \end{bmatrix} \right\|_2$, 要求:

$$\text{residual_norm}(x + t \cdot \text{dx}, v + t \cdot \text{dv}) \leq (1 - \alpha \cdot t) \cdot \text{current_residual}$$

同时保证 $x + t \cdot \text{dx} > 0$, 其中 $\alpha = 0.1$, $\beta = 0.5$ 为线搜索参数, current_residual 是当前迭代的残量范数 (对应代码中线搜索循环部分)。

当残量范数 $\text{current_residual} < \text{tol}$ ($\text{tol} = 10^{-8}$) 时, 停止迭代 (对应代码中迭代终止判断部分)。

2.3. 对偶 Newton 方法

通过求解对偶问题来间接得到原问题的解, 利用对偶函数、对偶梯度和对偶 Hessian 构建牛顿步长进行迭代。

对偶函数为 $g(\lambda) = \sum_{i=1}^n \exp(-A^T \lambda - 1) + b^T \lambda$, 其梯度 $\nabla g(\lambda)$ 满足 $\nabla g(\lambda) = -A \exp(-A^T \lambda - 1) + b$, Hessian 矩阵 $\nabla^2 g(\lambda)$ 为 $A \cdot \text{diag}(\exp(-A^T \lambda - 1)) \cdot A^T$ (对应代码中 `dual_function`、`dual_gradient`、`dual_hessian` 函数)。

对 Hessian 矩阵 $\nabla^2 g(\lambda)$ 进行 Cholesky 分解 $LL^T = \nabla^2 g(\lambda)$:

1) 若分解成功, 牛顿步长 $\text{d}\lambda$ 满足 $\text{d}\lambda = -(L^T)^{-1} L^{-1} \nabla g(\lambda)$

2) 若 Hessian 非正定导致 Cholesky 分解失败, 则对 Hessian 进行正则化 $H_{\text{reg}} = \nabla^2 g(\lambda) + 10^{-6} I$, 然后求解 $H_{\text{reg}} \cdot \text{d}\lambda = -\nabla g(\lambda)$ 得到步长 (对应代码中求解牛顿步长的部分)。

采用回溯线搜索确定步长 t , 满足

$$g(\lambda + t \cdot \text{d}\lambda) \leq g(\lambda) + \alpha \cdot t \cdot \nabla g(\lambda)^T \cdot \text{d}\lambda$$

其中 $\alpha = 0.3$, $\beta = 0.5$ 为线搜索参数, 迭代更新 $t = t \cdot \beta$ 直到满足条件或 t 过小 (对应代码中线搜索循环部分)。

当梯度范数 $\|\nabla g(\lambda)\| < \text{tol}$ 且 $\|\lambda - \lambda_{\text{prev}}\| < \text{tol}$ ($\text{tol} = 10^{-8}$, λ_{prev} 为上一次迭代的拉格朗日乘子) 时, 停止迭代。最后由对偶解 λ 得到原问题解 $x_{\text{dual}} = \exp(-A^T \lambda - 1)$ (对应代码中迭代终止判断和得到原问题解的部分)。

表 1: 函数及其作用整理

函数名称	作用
objective	计算目标函数值，目标函数为 $\sum(x_i \cdot \log(x_i))$ ，用于熵最大化等优化问题中衡量解的“优劣”
gradient	计算目标函数的梯度，梯度为 $\log(x_i) + 1$ ，用于牛顿类方法中确定搜索方向时计算梯度信息
hessian	计算目标函数的 Hessian 矩阵，Hessian 矩阵为 $\text{diag}(1/x_i)$ ，在牛顿方法里构建 KKT 系统等会用到二阶导数信息
constraint_violation	计算约束违反程度，即 $\ Ax - b\ _2$ ，用于判断解是否满足约束条件以及衡量优化过程中约束满足情况的变化
standard_newton_method	实现标准牛顿方法，通过构建并求解 KKT 系统，结合线搜索确定步长，迭代求解带约束的优化问题，记录优化过程中的目标函数值、梯度范数、约束违反程度等信息
infeasible_newton_method	实现不可行初始点牛顿方法，基于不可行初始点，通过构建残量相关的 KKT 系统，结合线搜索更新变量和拉格朗日乘子，迭代求解优化问题，同样记录优化过程数据
dual_function	计算对偶函数值，用于对偶牛顿方法中，对偶函数基于原问题构造，形式为 $\sum(\exp(-A^T @ \lambda_- - 1)) + b^T @ \lambda_-$
dual_gradient	计算对偶函数的梯度，梯度为 $-A @ \exp(-A^T @ \lambda_- - 1) + b$ ，在对偶牛顿方法中用于确定搜索方向
dual_hessian	计算对偶函数的 Hessian 矩阵，为 $A @ \text{diag}(\exp(-A^T @ \lambda_- - 1)) @ A^T$ ，在对偶牛顿方法里用于构建牛顿步长所需的矩阵
dual_newton_method	实现对偶牛顿方法，基于对偶函数，通过求解对偶问题的牛顿步长（利用 Cholesky 分解或正则化处理非正定 Hessian 情况），结合线搜索迭代求解，最终得到原问题的解并记录优化过程数据

表 2: 数据文件保存说明

文件名	说明
matrix_A.txt	约束矩阵 A
vector_b.txt	右侧向量 b
initial_x_hat.txt	初始点 $\hat{x}$
solution_*.txt	各种方法得到的最优解（‘*’ 代表不同方法的标识）
lambda_*.txt	各种方法得到的拉格朗日乘子（‘*’ 代表不同方法的标识）
comparison_results.txt	包含目标函数值、解差异、可行性检查等详细比较数据
complete_results.npy	包含所有结果的字典，便于后续数据分析与复用

3. 结果与比较

表 3: 各方法的最优目标函数值比较

方法	最优目标函数值
a. 标准 Newton 方法	-3.32809856×10^1
b1. 不可行 Newton 方法 ($x^{(0)} = \hat{x}$)	-3.32809856×10^1
b2. 不可行 Newton 方法 ($x^{(0)} = \mathbf{1}$)	-3.32809856×10^1
c. 对偶 Newton 方法	-3.32809856×10^1

表 4: 不同方法得到的解之间的差异

比较项	差异范数
$\ x_a - x_{b1}\ $	1.20×10^{-13}
$\ x_a - x_{b2}\ $	1.20×10^{-13}
$\ x_a - x_c\ $	1.20×10^{-13}
$\ x_{b1} - x_c\ $	1.06×10^{-15}

表 5: 不同方法得到的 Lagrange 乘子之间的差异

比较项	差异范数
$\ \lambda_a - \lambda_{b1}\ $	4.02×10^{-16}
$\ \lambda_a - \lambda_c\ $	4.55×10^{-16}
$\ \lambda_{b1} - \lambda_c\ $	2.60×10^{-16}

表 6: 各方法的约束满足情况

方法	约束违反度 $\ Ax - b\ $
标准 Newton 方法	5.80×10^{-15}
不可行 Newton 方法 ($x^{(0)} = \hat{x}$)	6.17×10^{-15}
不可行 Newton 方法 ($x^{(0)} = \mathbf{1}$)	7.91×10^{-15}
对偶 Newton 方法	8.57×10^{-15}

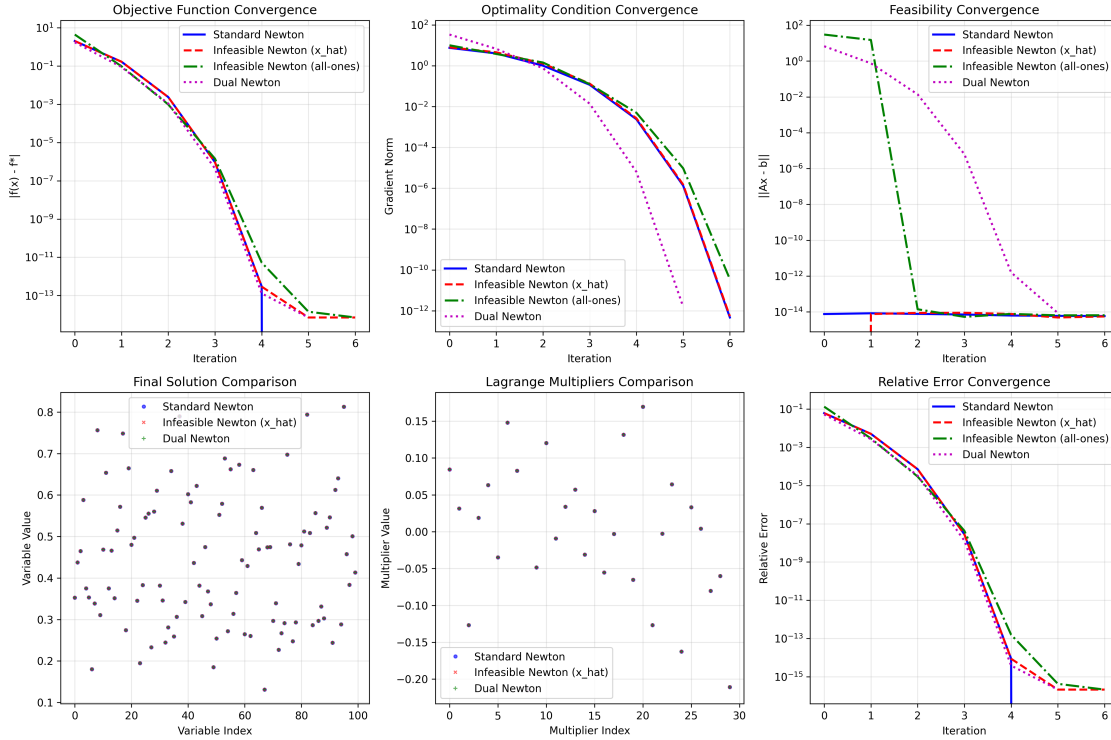
表 7: 各方法的迭代次数比较

方法	收敛所需迭代次数
标准 Newton 方法	7
不可行 Newton 方法 ($x^{(0)} = \hat{x}$)	7
不可行 Newton 方法 ($x^{(0)} = \mathbf{1}$)	7
对偶 Newton 方法	6

表 8: 各方法每次迭代的计算复杂度比较

方法	主要计算步骤	复杂度
标准 Newton 方法	- KKT 系统规模: $(n + p) \times (n + p)$ - Hessian 矩阵构造 + 线性系统求解	$O((n + p)^3)$
不可行 Newton 方法	- KKT 系统规模: $(n + p) \times (n + p)$ - Hessian 矩阵构造 + 线性系统求解 + 残差计算	$O((n + p)^3)$
对偶 Newton 方法	- Hessian 系统规模: $p \times p$ - 对偶 Hessian 构造 + 线性系统求解	$O(p^3 + np^2)$

图 1: 不同方法的结果对比



4. 结果分析

4.1. 解的一致性

所有方法均收敛到数值上相同的原始解和 Lagrange 乘子, 验证了这些方法在理论上的等价性。

4.2. 收敛行为

1) 从可行点开始, 不可行 Newton 方法的 Newton 步会保持可行性, 并且求解的系统与标准 Newton 方法的系统一致;

2) 不可行 Newton 方法即使从不可行点 ($x^{(0)} = \mathbf{1}$) 开始也能成功收敛;

3) 对偶 Newton 方法收敛所需的迭代次数更少 (6 次), 且数值稳定性更好。

4.3 计算效率

1) 对于 $n = 100$, $p = 30$, 对偶方法将问题复杂度从 $O((130)^3) = O(2.2 \times 10^6)$ 降低到 $O(30^3 + 100 \times 30^2) = O(117,000)$;

2) 当 n 相对于 p 增大时, 这种优势更加明显。

4.4. 数值精度

所有方法均达到机器精度级别 ($\sim 10^{-15}$) 的约束满足和最优性。

5. 完整代码

Listing 1: requirements.txt

```
1 numpy
2 scipy
3 matplotlib
4 pandas
5 seaborn
```

Listing 2: run.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import minimize
4 import os
5
6 os.makedirs('result', exist_ok=True)
7 n = 100
8 p = 30
9 np.random.seed(42)
10 A = np.random.randn(p, n)
11 rank_A = np.linalg.matrix_rank(A)
12 print(f"Matrix A rank: {rank_A} (full rank: {rank_A == p})")
13 x_hat = np.random.rand(n)
14 b = A @ x_hat
15 print(f"Problem dimensions: n={n}, p={p}")
16 print(f"Initial point x_hat range: [{x_hat.min():.4f}, {x_hat.max():.4f}]")
17
```



```

18 np.savetxt('result/matrix_A.txt', A, fmt='%.6f', header=f'Constraint matrix A ({p}x{n
    })')
19 np.savetxt('result/vector_b.txt', b, fmt='%.6f', header='Right-hand side vector b')
20 np.savetxt('result/initial_x_hat.txt', x_hat, fmt='%.6f', header='Initial point x_hat'
    )
21
22 class OptimizationLogger:
23     """Optimization progress logger"""
24     def __init__(self, name):
25         self.name = name
26         self.objectives = []
27         self.grad_norms = []
28         self.constraint_violations = []
29         self.iterations = []
30
31     def log(self, iteration, objective, grad_norm, constraint_violation):
32         self.iterations.append(iteration)
33         self.objectives.append(objective)
34         self.grad_norms.append(grad_norm)
35         self.constraint_violations.append(constraint_violation)
36
37     def objective(x):
38         """Objective function:  $\sum(x_i * \log(x_i))$ """
39         return np.sum(x * np.log(x))
40
41     def gradient(x):
42         """Objective gradient:  $\log(x_i) + 1$ """
43         return np.log(x) + 1
44
45     def hessian(x):
46         """Objective Hessian:  $\text{diag}(1/x_i)$ """
47         return np.diag(1 / x)
48
49     def constraint_violation(x, A, b):
50         """Constraint violation:  $\|Ax - b\|_2$ """
51         return np.linalg.norm(A @ x - b)
52
53     def standard_newton_method(A, b, x0, max_iter=100, tol=1e-8):
54         """Standard Newton method"""
55         logger = OptimizationLogger("Standard Newton")
56         x = x0.copy()
57         n = len(x)
58         p = A.shape[0]
59         for i in range(max_iter):
60             f_val = objective(x)
61             g = gradient(x)

```

```

62     H = hessian(x)
63     KKT_matrix = np.block([
64         [H, A.T],
65         [A, np.zeros((p, p))]
66     ])
67     rhs = np.concatenate([-g, b - A @ x])
68     try:
69         step = np.linalg.solve(KKT_matrix, rhs)
70     except np.linalg.LinAlgError:
71         step = np.linalg.lstsq(KKT_matrix, rhs, rcond=None)[0]
72     dx = step[:n]
73     lambda_vals = step[n:]
74     t = 1.0
75     alpha = 0.3
76     beta = 0.5
77     while np.any(x + t * dx <= 0) or objective(x + t * dx) > f_val + alpha * t * g
78         @ dx:
79         t *= beta
80         if t < 1e-12:
81             break
82     x = x + t * dx
83     grad_norm = np.linalg.norm(g + A.T @ lambda_vals)
84     constr_viol = constraint_violation(x, A, b)
85     logger.log(i, objective(x), grad_norm, constr_viol)
86     if grad_norm < tol and constr_viol < tol:
87         break
88     return x, lambda_vals, logger
89
90 def infeasible_newton_method(A, b, x0, max_iter=100, tol=1e-8):
91     """Infeasible start Newton method"""
92     logger = OptimizationLogger("Infeasible Newton")
93     x = x0.copy()
94     n = len(x)
95     p = A.shape[0]
96     v = np.zeros(p)
97     for i in range(max_iter):
98         r_dual = gradient(x) + A.T @ v
99         r_pri = A @ x - b
100         H = hessian(x)
101         KKT_matrix = np.block([
102             [H, A.T],
103             [A, np.zeros((p, p))]
104         ])
105         rhs = -np.concatenate([r_dual, r_pri])
106         try:
107             step = np.linalg.solve(KKT_matrix, rhs)

```

```

107     except np.linalg.LinAlgError:
108         step = np.linalg.lstsq(KKT_matrix, rhs, rcond=None)[0]
109     dx = step[:n]
110     dv = step[n:]
111     t = 1.0
112     alpha = 0.1
113     beta = 0.5
114
115     def residual_norm(x, v):
116         r_dual = gradient(x) + A.T @ v
117         r_pri = A @ x - b
118         return np.linalg.norm(np.concatenate([r_dual, r_pri]))
119
120     current_residual = residual_norm(x, v)
121     while np.any(x + t * dx <= 0) or residual_norm(x + t * dx, v + t * dv) > (1 -
122         alpha * t) * current_residual:
123         t *= beta
124         if t < 1e-12:
125             break
126     x = x + t * dx
127     v = v + t * dv
128     grad_norm = np.linalg.norm(r_dual)
129     constr_viol = np.linalg.norm(r_pri)
130     logger.log(i, objective(x), grad_norm, constr_viol)
131     if current_residual < tol:
132         break
133     return x, v, logger
134
135 def dual_function(lambda_, A, b):
136     """Dual function for entropy maximization"""
137     return np.sum(np.exp(-A.T @ lambda_ - 1)) + b.T @ lambda_
138
139 def dual_gradient(lambda_, A, b):
140     """Gradient of the dual function"""
141     return -A @ np.exp(-A.T @ lambda_ - 1) + b
142
143 def dual_hessian(lambda_, A):
144     """Hessian of the dual function"""
145     exp_term = np.exp(-A.T @ lambda_ - 1)
146     return A @ np.diag(exp_term) @ A.T
147
148 def dual_newton_method(A, b, lambda0, max_iter=100, tol=1e-8):
149     """Dual Newton method - CORRECTED"""
150     logger = OptimizationLogger("Dual Newton")
151     lambda_ = lambda0.copy()
152     for i in range(max_iter):

```

```

152     f_dual = dual_function(lambda_, A, b)
153     g_dual = dual_gradient(lambda_, A, b)
154     H_dual = dual_hessian(lambda_, A)
155     try:
156         L = np.linalg.cholesky(H_dual)
157         d_lambda = -np.linalg.solve(L.T, np.linalg.solve(L, g_dual))
158     except np.linalg.LinAlgError:
159         print(f"Iteration {i}: Hessian not positive definite, using regular solve
160               with damping")
161         H_reg = H_dual + 1e-6 * np.eye(H_dual.shape[0])
162         d_lambda = -np.linalg.solve(H_reg, g_dual)
163     t = 1.0
164     alpha = 0.3
165     beta = 0.5
166     while dual_function(lambda_ + t * d_lambda, A, b) > f_dual + alpha * t *
167           g_dual.T @ d_lambda:
168         t *= beta
169         if t < 1e-12:
170             print(f"Line search failed at iteration {i}")
171             break
172         lambda_prev = lambda_.copy()
173         lambda_ = lambda_ + t * d_lambda
174         x_dual = np.exp(-A.T @ lambda_ - 1)
175         grad_norm = np.linalg.norm(g_dual)
176         constr_viol = constraint_violation(x_dual, A, b)
177         logger.log(i, objective(x_dual), grad_norm, constr_viol)
178         if grad_norm < tol and np.linalg.norm(lambda_ - lambda_prev) < tol:
179             break
180     x_dual = np.exp(-A.T @ lambda_ - 1)
181     return x_dual, lambda_, logger
182
183 print("\nRunning optimization algorithms...")
184 print("1. Standard Newton method...")
185 x_a, lambda_a, logger_a = standard_newton_method(A, b, x_hat)
186 print("2. Infeasible Newton method (initial: x_hat)...")
187 x_b1, lambda_b1, logger_b1 = infeasible_newton_method(A, b, x_hat)
188 print("3. Infeasible Newton method (initial: all-ones)...")
189 x_b2, lambda_b2, logger_b2 = infeasible_newton_method(A, b, np.ones(n))
190 print("4. Dual Newton method...")
191 lambda0 = lambda_a.copy() + 0.1 * np.random.randn(p)
192 x_c, lambda_c, logger_c = dual_newton_method(A, b, lambda0)
193
194 np.savetxt('result/solution_standard_newton.txt', x_a, fmt='%.8f', header='Optimal
195           solution - Standard Newton')
196 np.savetxt('result/solution_infeasible_newton_xhat.txt', x_b1, fmt='%.8f', header='
197           Optimal solution - Infeasible Newton (x_hat)')

```

```

194 np.savetxt('result/solution_infeasible_newton_ones.txt', x_b2, fmt='%.8f', header='
    Optimal solution - Infeasible Newton (all-ones)')
195 np.savetxt('result/solution_dual_newton.txt', x_c, fmt='%.8f', header='Optimal
    solution - Dual Newton')
196
197 np.savetxt('result/lambda_standard_newton.txt', lambda_a, fmt='%.8f', header='Lagrange
    multipliers - Standard Newton')
198 np.savetxt('result/lambda_infeasible_newton_xhat.txt', lambda_b1, fmt='%.8f', header='
    Lagrange multipliers - Infeasible Newton (x_hat)')
199 np.savetxt('result/lambda_dual_newton.txt', lambda_c, fmt='%.8f', header='Lagrange
    multipliers - Dual Newton')
200
201 print("\n" + "="*50)
202 print("Results Comparison")
203 print("="*50)
204 obj_a = objective(x_a)
205 obj_b1 = objective(x_b1)
206 obj_b2 = objective(x_b2)
207 obj_c = objective(x_c)
208 print(f"\nOptimal objective values:")
209 print(f"Standard Newton: {obj_a:.8e}")
210 print(f"Infeasible Newton (x_hat): {obj_b1:.8e}")
211 print(f"Infeasible Newton (all-ones): {obj_b2:.8e}")
212 print(f"Dual Newton: {obj_c:.8e}")
213 print(f"\nSolution differences:")
214 print(f"||x_a - x_b1|| = {np.linalg.norm(x_a - x_b1):.2e}")
215 print(f"||x_a - x_b2|| = {np.linalg.norm(x_a - x_b2):.2e}")
216 print(f"||x_a - x_c|| = {np.linalg.norm(x_a - x_c):.2e}")
217 print(f"||x_b1 - x_c|| = {np.linalg.norm(x_b1 - x_c):.2e}")
218 print(f"\nLagrange multiplier differences:")
219 print(f"||_a - _b1|| = {np.linalg.norm(lambda_a - lambda_b1):.2e}")
220 print(f"||_a - _c|| = {np.linalg.norm(lambda_a - lambda_c):.2e}")
221 print(f"||_b1 - _c|| = {np.linalg.norm(lambda_b1 - lambda_c):.2e}")
222 print(f"\nFeasibility check (||Ax - b||):")
223 print(f"Standard Newton: {constraint_violation(x_a, A, b):.2e}")
224 print(f"Infeasible Newton (x_hat): {constraint_violation(x_b1, A, b):.2e}")
225 print(f"Infeasible Newton (all-ones): {constraint_violation(x_b2, A, b):.2e}")
226 print(f"Dual Newton: {constraint_violation(x_c, A, b):.2e}")
227
228 with open('result/comparison_results.txt', 'w') as f:
229     f.write("OPTIMIZATION RESULTS COMPARISON\n")
230     f.write("="*50 + "\n")
231     f.write(f"Problem dimensions: n={n}, p={p}\n")
232     f.write(f"Matrix A rank: {rank_A} (full rank: {rank_A == p})\n\n")
233
234     f.write("Optimal objective values:\n")

```

```

235     f.write(f"Standard Newton: {obj_a:.8e}\n")
236     f.write(f"Infeasible Newton (x_hat): {obj_b1:.8e}\n")
237     f.write(f"Infeasible Newton (all-ones): {obj_b2:.8e}\n")
238     f.write(f"Dual Newton: {obj_c:.8e}\n\n")
239
240     f.write("Solution differences:\n")
241     f.write(f"||x_a - x_b1|| = {np.linalg.norm(x_a - x_b1):.2e}\n")
242     f.write(f"||x_a - x_b2|| = {np.linalg.norm(x_a - x_b2):.2e}\n")
243     f.write(f"||x_a - x_c|| = {np.linalg.norm(x_a - x_c):.2e}\n")
244     f.write(f"||x_b1 - x_c|| = {np.linalg.norm(x_b1 - x_c):.2e}\n\n")
245
246     f.write("Feasibility check (||Ax - b||):\n")
247     f.write(f"Standard Newton: {constraint_violation(x_a, A, b):.2e}\n")
248     f.write(f"Infeasible Newton (x_hat): {constraint_violation(x_b1, A, b):.2e}\n")
249     f.write(f"Infeasible Newton (all-ones): {constraint_violation(x_b2, A, b):.2e}\n")
250     f.write(f"Dual Newton: {constraint_violation(x_c, A, b):.2e}\n\n")
251
252     f.write("Iteration counts:\n")
253     f.write(f"Standard Newton method: {len(logger_a.iterations)}\n")
254     f.write(f"Infeasible Newton (x_hat): {len(logger_b1.iterations)}\n")
255     f.write(f"Infeasible Newton (all-ones): {len(logger_b2.iterations)}\n")
256     f.write(f"Dual Newton method: {len(logger_c.iterations)}\n")
257
258     plt.figure(figsize=(15, 10))
259     plt.subplot(2, 3, 1)
260     plt.semilogy(logger_a.iterations, [abs(obj - obj_a) for obj in logger_a.objectives],
261                  'b-', linewidth=2, label='Standard Newton')
262     plt.semilogy(logger_b1.iterations, [abs(obj - obj_a) for obj in logger_b1.objectives],
263                  'r--', linewidth=2, label='Infeasible Newton (x_hat)')
264     plt.semilogy(logger_b2.iterations, [abs(obj - obj_a) for obj in logger_b2.objectives],
265                  'g-.', linewidth=2, label='Infeasible Newton (all-ones)')
266     plt.semilogy(logger_c.iterations, [abs(obj - obj_a) for obj in logger_c.objectives],
267                  'm:', linewidth=2, label='Dual Newton')
268     plt.xlabel('Iteration')
269     plt.ylabel('|f(x) - f*|')
270     plt.title('Objective Function Convergence')
271     plt.legend()
272     plt.grid(True, alpha=0.3)
273
274     plt.subplot(2, 3, 2)
275     plt.semilogy(logger_a.iterations, logger_a.grad_norms, 'b-', linewidth=2, label='
Standard Newton')
276     plt.semilogy(logger_b1.iterations, logger_b1.grad_norms, 'r--', linewidth=2, label='
Infeasible Newton (x_hat)')
277     plt.semilogy(logger_b2.iterations, logger_b2.grad_norms, 'g-.', linewidth=2, label='
Infeasible Newton (all-ones)')

```

```

278 plt.semilogy(logger_c.iterations, logger_c.grad_norms, 'm:', linewidth=2, label='Dual
    Newton')
279 plt.xlabel('Iteration')
280 plt.ylabel('Gradient Norm')
281 plt.title('Optimality Condition Convergence')
282 plt.legend()
283 plt.grid(True, alpha=0.3)
284
285 plt.subplot(2, 3, 3)
286 plt.semilogy(logger_a.iterations, logger_a.constraint_violations, 'b-', linewidth=2,
    label='Standard Newton')
287 plt.semilogy(logger_b1.iterations, logger_b1.constraint_violations, 'r--', linewidth
    =2, label='Infeasible Newton (x_hat)')
288 plt.semilogy(logger_b2.iterations, logger_b2.constraint_violations, 'g-.', linewidth
    =2, label='Infeasible Newton (all-ones)')
289 plt.semilogy(logger_c.iterations, logger_c.constraint_violations, 'm:', linewidth=2,
    label='Dual Newton')
290 plt.xlabel('Iteration')
291 plt.ylabel('||Ax - b||')
292 plt.title('Feasibility Convergence')
293 plt.legend()
294 plt.grid(True, alpha=0.3)
295
296 plt.subplot(2, 3, 4)
297 plt.plot(x_a, 'bo', markersize=3, alpha=0.6, label='Standard Newton')
298 plt.plot(x_b1, 'rx', markersize=3, alpha=0.6, label='Infeasible Newton (x_hat)')
299 plt.plot(x_c, 'g+', markersize=4, alpha=0.6, label='Dual Newton')
300 plt.xlabel('Variable Index')
301 plt.ylabel('Variable Value')
302 plt.title('Final Solution Comparison')
303 plt.legend()
304 plt.grid(True, alpha=0.3)
305
306 plt.subplot(2, 3, 5)
307 plt.plot(lambda_a, 'bo', markersize=3, alpha=0.6, label='Standard Newton')
308 plt.plot(lambda_b1, 'rx', markersize=3, alpha=0.6, label='Infeasible Newton (x_hat)')
309 plt.plot(lambda_c, 'g+', markersize=4, alpha=0.6, label='Dual Newton')
310 plt.xlabel('Multiplier Index')
311 plt.ylabel('Multiplier Value')
312 plt.title('Lagrange Multipliers Comparison')
313 plt.legend()
314 plt.grid(True, alpha=0.3)
315
316 plt.subplot(2, 3, 6)
317 rel_error_a = [abs(obj - obj_a)/(abs(obj_a)+1e-10) for obj in logger_a.objectives]
318 rel_error_b1 = [abs(obj - obj_a)/(abs(obj_a)+1e-10) for obj in logger_b1.objectives]

```

```

319 rel_error_b2 = [abs(obj - obj_a)/(abs(obj_a)+1e-10) for obj in logger_b2.objectives]
320 rel_error_c = [abs(obj - obj_a)/(abs(obj_a)+1e-10) for obj in logger_c.objectives]
321 plt.semilogy(logger_a.iterations, rel_error_a, 'b-', linewidth=2, label='Standard
    Newton')
322 plt.semilogy(logger_b1.iterations, rel_error_b1, 'r--', linewidth=2, label='Infeasible
    Newton (x_hat)')
323 plt.semilogy(logger_b2.iterations, rel_error_b2, 'g-.', linewidth=2, label='Infeasible
    Newton (all-ones)')
324 plt.semilogy(logger_c.iterations, rel_error_c, 'm:', linewidth=2, label='Dual Newton')
325 plt.xlabel('Iteration')
326 plt.ylabel('Relative Error')
327 plt.title('Relative Error Convergence')
328 plt.legend()
329 plt.grid(True, alpha=0.3)
330
331 plt.tight_layout()
332 plt.savefig('result/optimization_comparison.png', dpi=300, bbox_inches='tight')
333 plt.show()
334
335 print("\n" + "="*50)
336 print("Computational Complexity Analysis")
337 print("="*50)
338 print(f"\nIteration counts:")
339 print(f"Standard Newton method: {len(logger_a.iterations)}")
340 print(f"Infeasible Newton (x_hat): {len(logger_b1.iterations)}")
341 print(f"Infeasible Newton (all-ones): {len(logger_b2.iterations)}")
342 print(f"Dual Newton method: {len(logger_c.iterations)}")
343 n, p = A.shape
344 print(f"\nProblem dimensions: n={n}, p={p}")
345
346 results = {
347     'x_optimal': x_a,
348     'lambda_optimal': lambda_a,
349     'objective_value': obj_a,
350     'A': A,
351     'b': b,
352     'x_hat': x_hat,
353     'solution_standard': x_a,
354     'solution_infeasible_xhat': x_b1,
355     'solution_infeasible_ones': x_b2,
356     'solution_dual': x_c,
357     'lambda_standard': lambda_a,
358     'lambda_infeasible_xhat': lambda_b1,
359     'lambda_dual': lambda_c
360 }
361

```



```
362 np.save('result/complete_results.npy', results)
363 print(f"\nResults saved to 'result' directory")
364 print("Files generated:")
365 print("- matrix_A.txt: Constraint matrix A")
366 print("- vector_b.txt: Right-hand side vector b")
367 print("- initial_x_hat.txt: Initial point")
368 print("- solution_*.txt: Optimal solutions from each method")
369 print("- lambda_*.txt: Lagrange multipliers from each method")
370 print("- comparison_results.txt: Detailed comparison of results")
371 print("- optimization_comparison.png: Convergence plots")
372 print("- complete_results.npy: Complete results dictionary")
373 print("\nOptimization completed!")
```
